

Hash Functions and Hashing

Trading a Search for an Address Calculation

Unit III - Lecture 28

Hash function key $\rightarrow$ index, $O(1)$ to compute

Chaining one bucket holds a small list

Linear probing one bucket, find the next free slot

The problem: can searching be even faster?

Binary search needs $O(\log n)$ comparisons because it must NARROW DOWN where a key could be.

What if, instead of searching, we could CALCULATE exactly where a key belongs - in one step, every time?

- An array already does this for POSITION: `arr[5]` is found in $O(1)$, because 5 IS the address.
- Hashing extends the same idea to arbitrary keys: turn the key itself into an array index using a FORMULA, not a search.
- The formula is called a HASH FUNCTION.

The division method: the simplest hash function

$$h(\text{key}) = \text{key} \% \text{tableSize}$$

key	23	12	17	9	24	15
$\text{key} \% 7$	2	5	3	2	3	1

- $\text{tableSize} = 7$ here, so every key maps to an index between 0 and 6.
- This runs in $O(1)$ - a single modulo operation, no comparisons, no searching at all.
- Notice: 23 and 9 both map to index 2. 17 and 24 both map to index 3.

Collisions are not a bug - they are guaranteed

The pigeonhole principle

If there are more possible KEYS than table SLOTS - which is almost always true - then by simple counting, at least two keys must eventually hash to the same slot. This is called a COLLISION.

- A hash function's job is not to AVOID collisions (usually impossible) but to SPREAD keys evenly, so collisions stay rare.
- What matters just as much: a clear, agreed RULE for what to do when a collision happens.
- Two standard rules follow: chaining, and open addressing (linear probing).

** With $tableSize = 7$, could a PERFECT hash function that never collides exist for an unlimited stream of integer keys?*

Separate chaining: each slot holds a small list

Instead of one VALUE per slot, each slot holds the HEAD of a linked list. A collision just means another node joins that slot's list.

- This is exactly Lecture 20's array-of-lists idea, and exactly Lecture 15's linked list - nothing new is invented here, only combined.
- `insert(key)`: compute `h(key)`, then `insertFront` onto that slot's list - $O(1)$, same discipline as every linked-list `insertFront` in this course.
- `search(key)`: compute `h(key)`, then walk ONLY that slot's list - never the whole table.

Chaining: code and the verified result

```
void insert(int key)
{
    int idx = hashFunc(key);
    struct Node *node = malloc(sizeof *node);
    node->key = key;
    node->next = table[idx];
    table[idx] = node;
}
```

- Verified: 9 and 23 both live at index 2. 17 and 24 both live at index 3.
- insertFront means the MOST RECENT key at each index is first - 9 was inserted after 23, so 9 sits in front.
- search(9): 1 probe (it is first in its list). search(24): 1 probe (also first). Every OTHER search in this table also costs just 1 probe.

0	NULL
1	15 -> NULL
2	9 -> 23 -> NULL
3	24 -> 17 -> NULL
4	NULL
5	12 -> NULL
6	NULL

Linear probing: stay in the SAME array, find the next gap

No linked lists at all - every key lives directly in the array. On a collision, check the NEXT slot, then the next, until an empty one is found (wrapping around at the end).

- This is called OPEN ADDRESSING - the key's home slot is only a starting point, not a guarantee.
- No malloc, no pointers - just one plain array. Often faster in practice due to memory layout, at the cost of harder deletion (not covered today).
- The array can fill up completely - unlike chaining, which never truly runs out of room.

Linear probing: code and the verified result

```
int insert(int key)
{
    int idx = hashFunc(key);
    while (table[idx] != EMPTY)
        idx = (idx + 1) % TABLE_SIZE;
    table[idx] = key;
    return idx;
}
```

- 9's home is index 2, but 23 is already there - probes forward to index 4.
- 24's home is index 3, but 17 is there - probes forward through 4 (occupied by 9!) and 5 (occupied by 12) before landing at 6.

0	1	2	3	4	5	6
.	15	23	17	9	12	24

23,12,17: no collision - placed at home index

9: home=2 occupied -> probes to index 4 (2 hops)

24: home=3 occupied -> probes to index 6 (3 hops)

The weakness: collisions can compound

24 shares a home index with only ONE other key (17). Yet it needed THREE hops - because 9's earlier collision left a block of occupied slots (2,3,4) sitting directly in 24's path.

This is called SECONDARY CLUSTERING: one collision makes the next one worse, even between UNRELATED keys.

- Chaining does not suffer from this - each slot's list only ever grows from keys that TRULY share that home index.
- This is the central trade-off of the lecture: chaining is simpler to reason about; linear probing is more memory-efficient but can degrade unpredictably as the table fills up.

Complexity, and the number that predicts it: load factor

load factor (alpha) = number of keys / table size

Aspect	Chaining	Linear probing
Average case (small alpha)	$O(1)$	$O(1)$
Worst case	$O(n)$ - one long chain	$O(n)$ - long cluster
Can the table fill up?	No - lists grow forever	Yes - a fixed number of slots
This example's alpha	6 keys / 7 slots = 0.86	6 keys / 7 slots = 0.86

LECTURE 28 - SUMMARY

Calculate the address, do not search for it

- A hash function turns a key into an array index in $O(1)$ - the division method, $\text{key} \% \text{tableSize}$, is the simplest example.
- Collisions are guaranteed eventually (pigeonhole principle) - the real design question is how to handle them, not whether they happen.
- Chaining: each slot holds a linked list. Simple, never fills up, degrades gracefully.
- Linear probing: each slot holds one key directly. No extra memory, but can suffer SECONDARY CLUSTERING, verified directly in today's example.
- Average case for both is $O(1)$ when the load factor stays low - the entire promise of hashing depends on keeping α well under 1.

EXIT TICKET $\text{tableSize} = 5$. Insert keys 11, 6, 16, 3 using linear probing, in that order. Show the final table and identify any clustering.